ITU-MiniTwit

DevOps, Software Evolution and Software Maintenance
BSc Group F (gruppe 1)

Sebastian Holt Svendsen <sesv@itu.dk>
Andreas Nygaard Schiøtt <anys@itu.dk> Member Three <xxxx@itu.dk>
Member Four <xxxx@itu.dk> Member Five <xxxx@itu.dk>

May 9, 2026

Contents

1 System's Perspective **2**
1.1 Architecture and Design . . . 2
1.2 Dependencies . . . 2
1.3 Current State of the Systems . . . 2

2 Process' Perspective **3**
2.1 CI/CD Pipeline . . . 3
2.2 Monitoring . . . 5
2.3 Logging . . . 6
2.4 Security Hardening . . . 6
2.5 Availability and Scaling . . . 7

3 Reflection Perspective **8**
3.1 Evolution and Refactoring . . . 8
3.2 Operation . . . 8
3.3 Maintenance . . . 8
3.4 DevOps Style of Work . . . 8

4 Use of Generative AI **9**
4.1 Tools Used . . . 9
4.2 Impact on Work Process . . . 9
ITU-MiniTwit GitHub Repository

1 System's Perspective

Authors: TODO: names

1.1 Architecture and Design

1.2 Dependencies

Table 1: Technology dependencies

Technology	**Purpose**
Docker	Containerisation of all services

1.3 Current State of the Systems

2 Process' Perspective

Authors: Andreas Schiøtt

2.1 CI/CD Pipeline

Table 2: CI/CD tools

Technology	Purpose
GitHub Actions	CI/CD automation
Playwright	end-to-end testing
SonarCloud	Static analysis and code coverage
Hadolint	Dockerfile linter
ShellCheck	Shell script linter
Docker Build	Build and push container images to Docker Hub
Docker Swarm	Automated docker container orchestration
rsync	Idempotent Sync files to the remote swarm manager
Grafana	Logging / monitoring running Prometheus, loki, Alloy
Prometheus	Metric collection and monitoring
Loki	Log aggregation system
Grafana Alloy	Log and monitoring collection

Before we had a running cloud hosted droplet we only ran CI to run a docker container and run unit test on a locally hosted sqlite based application.

After cloud hosting on **DigitalOcean** was setup we added the CD pipeline to the CI pipeline. The CI/CD pipeline were both triggered concurrently on push to main branch which then builds a **Docker image**, pushes it to **Docker Hub** and then deploys it to the **DigitalOcean** droplet using **SSH & docker pull**. With the later focus on idempotent deployment, we added a check to see if the CI pipeline succeeded before the CD pipeline is triggered.

The CI pipeline was extended to include end-to-end testing using **Playwright**, and static code analysis using **SonarCloud**, **Hadolint** and **ShellCheck**. As shown in fig. 1, on every push the pipeline detects which paths changed, builds the project, and then runs static analysis, unit tests, SonarCloud analysis, and end-to-end tests in parallel.

While writing the report we itterativly extended the CI pipeline to check which files were changed to decicde which steps to run, if only the report directory was changed we only run "build PDF" report" step and vice versa.

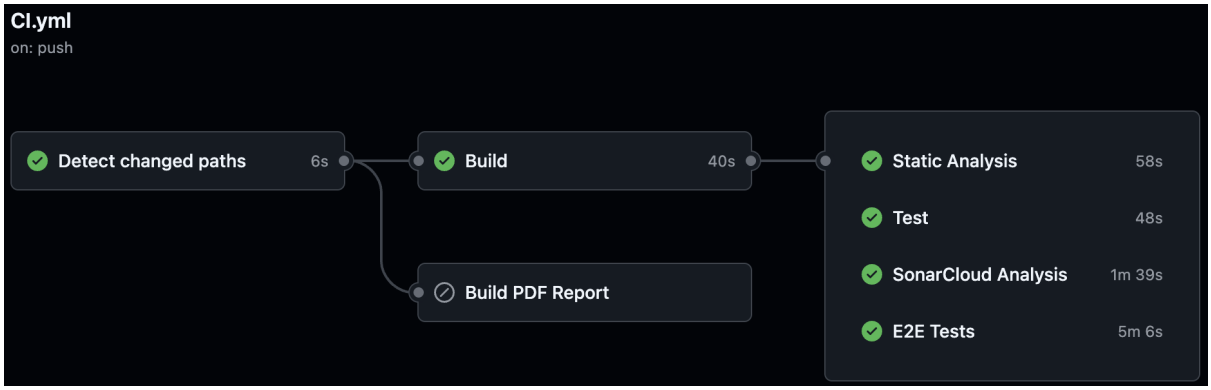

Figure 1: CI pipeline (`CI.yml`).

The CD pipeline is triggered only after a successful CI run, ensuring that broken builds are never deployed. As shown in fig. 2, it consists of three sequential jobs: building and pushing the Docker image (*build-push*), deploying the application (*Deploy*), and deploying the monitoring stack (*Deploy-Monitoring*) with **Grafana**, **Prometheus**, **Loki** and **Grafana Alloy**.

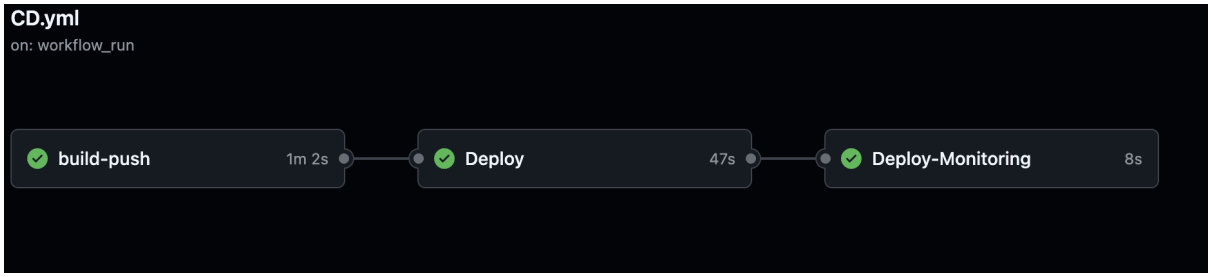

Figure 2: CD pipeline

To ensure high availability of the application, we extended the CD pipeline to deploy the application using **Docker Swarm**, which deploys the application as a swarm with multiple replica nodes while a load balencer distributes traffic across the replicas. this deployment uses rolling updates, which starts a new docker container before closing the previous one to avoid downtime. The application also uses shared **Redis** which caches data across the replicas. The detailed steps of the *Deploy* job are shown in fig. 3:

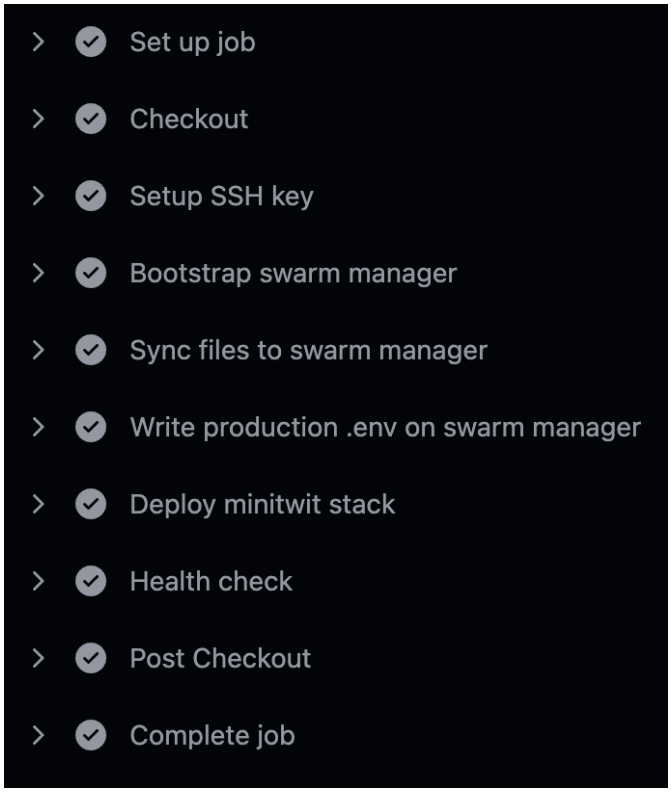

Figure 3: Detailed steps of the *Deploy* job.

2.2 Monitoring

Link to Grafana monitoring dashboard
Metrics are collected through **Prometheus** and visualised in **Grafana**, both running as Docker services on a dedicated monitoring droplet, isolated from the application. Prometheus scrapes application metrics via `'itu-minitwit-app'` defined in the Prometheus configuration and metrics from Node Exporter running on each Swarm node.

We have itterativly upated which data is collected and visualised in the Grafana dashboard based on the needs of the project. At the projects end we have the Grafana dashboard organised into five sections, as shown in fig. 4:

- **Health overview** — application up/down status, average response time, requests / sec, in-flight requests, error rate and current memory usage.
- **HTTP traffic** — request rate by status code, ... response-time percentiles grouped by p50 p90 and p99, request rate /s grouped by HTTP method, and the response time grouped by api endpoints.
- **Error tracking** error rates req/s grouped by 400/500 http response, errors mesured in req/s per endpoint.
- **Process & runtime** — process CPU usage, process memory usage, thread count, garbage colllection rate.
- **Uptime & Availability** — uptime & availability over time (bar graf), Availability mesured in percentage over the selected time period.

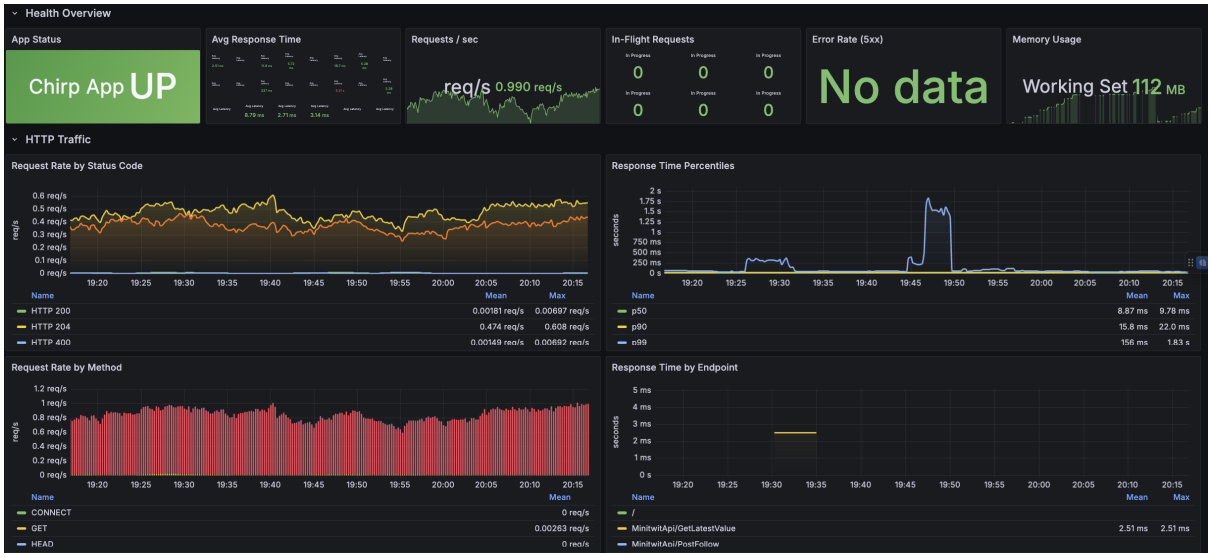

Figure 4: Grafana monitoring dashboard.

2.3 Logging

Link to Grafana logging dashboard

Application logs are aggregated using **Loki** and collected by **Grafana Alloy**. Each line in the logs is labelled with the container name, service name, stream, and image name. **Loki** rejects old samples above the "age" of 168hours (7 days) to prevent unbounded log storage growth.

The application logs the following events:

- HTTP requests — method, path, response status code, and elapsed time mesured in ms.
- Unhandled exceptions — method, path, error details.
- Custom logs through `ILogger` throughout the minitwit codebase.

2.4 Security Hardening

measures applied to harden the system:

- **GitHub Secrets** — database connection strings and API keys are stored as encrypted secrets and injected at deploy time.
- **Firewall** — inbound and outbound traffic on both application droplets and the database is restricted to only the ports and sources required.

Static code analysis is integrated in the CI pipeline using **SonarCloud**. Every build is analysed for bugs, coverage, code duplication and security vulnerabilities. As shown in fig. 5, there are still a number of security vulnerabilities which we unfortunately did not have time to fix.

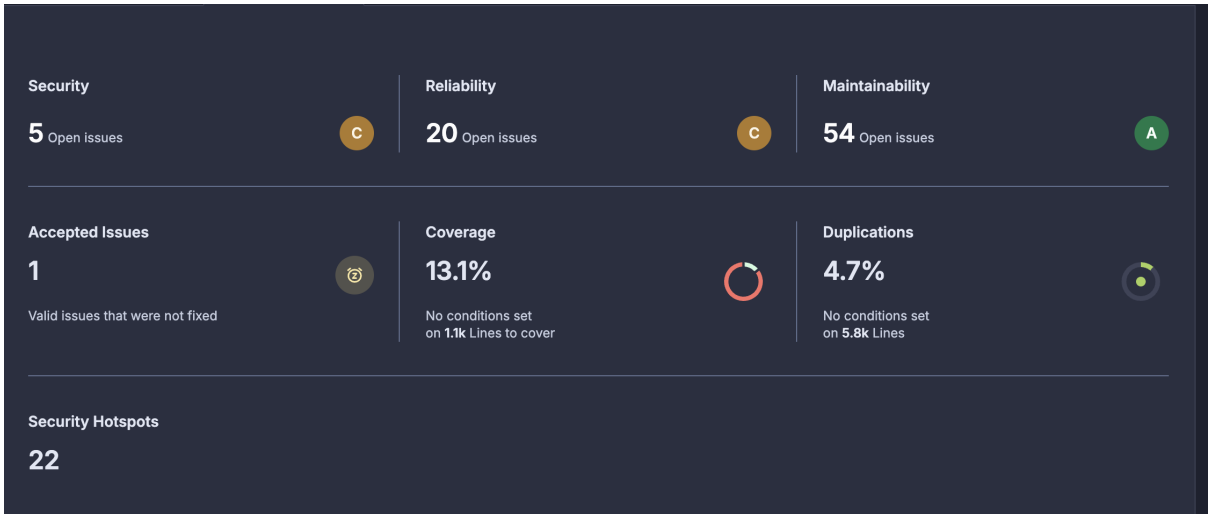

Figure 5: SonarCloud quality gate results.

2.5 Availability and Scaling

TODO : Insert pictures of stucture umls

The application is deployed as a **Docker Swarm** stack with **2 replicas** hosted as 2 seperate **DigtalOcean droplets**. our load balencer distributes traffic across replicas, and automatically reroutes requests away from any node that becomes unavailable/crash. Crashed containers are restarted automatically via the on-failure restart policy.

the application uses a **Redis** instance shared between all nodes which caches data between replica nodes, which ensures consistent returned data regardless of which replica handles the request.
Deployments are made using rolling updates, which starts a new docker container before closing the previous one to avoid downtime. this process is repeated for each node with a 10 second delay to ensure that at least one instance is always available for requests during deployments to reduced downtime of the system.

3 Reflection Perspective

Authors: TODO: names

3.1 Evolution and Refactoring

3.2 Operation

3.3 Maintenance

3.4 DevOps Style of Work

4 Use of Generative AI

Authors: TODO: names

4.1 Tools Used

4.2 Impact on Work Process

References